LABORATOR 6 - statistica

EX#1 Fie $\lambda \in (0, \infty)$. Creaţi un fişier în Python® prin care să se genereze un număr aleator X distribuit Poisson $Pois(\lambda)$

(a) folosind algoritmul de generare din Python®;

(b) $X = Y$, unde Y este un număr generat aleator uniform cu o distribuţie binomială $Bin(n, \lambda/n)$, pentru un $n \in \mathbb{N}$ suficient de mare.

Creaţi un fişier în Python® prin care

(c) să se realizeze N simulări pentru fiecare dintre cazurile (a), respectiv (b);

(d) să se afişeze histogramele corespunzătoare simulărilor realizate la (c) (pentru fiecare dintre cazurile (a), respectiv (b));

(e) să se afişeze graficul ponderilor $p_k := e^{-\lambda}\dfrac{\lambda^k}{k!}$, $k = \overline{0, n}$ pentru un $n \in \mathbb{N}$ suficient de mare;

(f) să se estimeze numeric media şi varianţa variabilei aleatoare distribuită Poisson $Pois(\lambda)$ folosind simulările de la (c) (pentru fiecare dintre cazurile (a), respectiv (b));

```
import numpy as np

# Parametru lambda
lam = 50 # exemplu, poţi schimba

# (a) Generare directă Poisson
X_a = np.random.poisson(lam)
print("Metoda directă Poisson:", X_a)

# (b) Aproximare prin binomială
n = 10000 # n suficient de mare
p = lam / n
Y = np.random.binomial(n, p)
print("Aproximare prin binomială:", Y)
```

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Simulări şi grafice pentru distribuţia Poisson
"""

import numpy as np
import matplotlib.pyplot as plt
from math import factorial, exp
from scipy.stats import poisson
#%% Parametri
N = int(1e7) # număr simulări
lam = 50 # parametru Poisson
n_bin = 1000 # n suficient de mare pentru aprox. binomială

#%% (c) Simulări
# Metoda (a) - Poisson direct
pois_sim = np.random.poisson(lam, size=N)
print(pois_sim)

# Metoda (b) - Aproximare prin Binomial
p_bin = lam / n_bin
binom_sim = np.random.binomial(n_bin, p_bin, size=N)
```

```
#%% (d) Histograme
plt.figure()
bins_a = np.max(pois_sim) - np.min(pois_sim) + 1
plt.hist(pois_sim, bins=bins_a)
plt.title("Histogramă simulări Poisson direct")
plt.xlabel("Valoare Poisson")
plt.ylabel("Frecvenţă")

plt.figure()
bins_b = np.max(binom_sim) - np.min(binom_sim) + 1
plt.hist(binom_sim, bins=bins_b)
plt.title("Histogramă simulări Poisson prin binomială")
plt.xlabel("Valoare Poisson aproximată")
plt.ylabel("Frecvenţă")


#%% (e) Grafic ponderi pk = e^-λ λ^k / k!
# valori k în jurul mediei (λ ± 4√λ)
k_vals = np.arange(0, 100)
p_k = poisson.pmf(k_vals, lam) # calculează ponderile corect
#adică probabilitatea ca o variabilă aleatoare Poisson(λ) să ia valoarea k.

plt.figure()
plt.plot(k_vals, p_k, 'o', label='ponderi pk')
plt.title('Pois(50)')
plt.xlabel('k')
```

```
plt.ylabel('p_k')
plt.legend()
plt.grid(True)
plt.show()

#%% (f) Estimare medie şi varianţă
# Poisson direct
mean_pois = np.mean(pois_sim)
var_pois = np.var(pois_sim)
print(f"Poisson direct: media ≈ {mean_pois:.3f}, varianţa ≈ {var_pois:.3f}")

# Aproximare binomială
mean_binom = np.mean(binom_sim)
var_binom = np.var(binom_sim)
print(f"Poisson prin binomială: media ≈ {mean_binom:.3f}, varianţa ≈ {var_binom:.3f}")
```

EX#2 La un spital ajung în medie 150 pacienţi într-un interval de o oră. Creaţi un fişier în Python® prin care să se determine probabilitatea să ajungă mai mult de 150 pacienţi într-o oră.

```
from scipy.stats import poisson

# Parametru Poisson
```

```
lam = 150  # media (λ)

# Probabilitatea ca X > 150
p = 1 - poisson.cdf(150, lam)

print(f"Probabilitatea ca numărul de pacienţi să fie >150 este {p:.4f}")

#Probabilitatea ca numărul de pacienţi să fie >150 este 0.4783


#poisson.pmf(k, λ) = probabilitatea exactă pentru k pacienţi.
#poisson.cdf(k, λ) = probabilitatea cumulată până la k
```

EX#3 Fie $\lambda \in (0, \infty)$. Creaţi un fişier în Python® prin care să se genereze un număr aleator X distribuit exponenţial $Exp(\lambda)$

(a) folosind algoritmul de generare din Python®;

(b) $X = -\frac{1}{\lambda}\ln(U)$, unde U este un număr generat aleator uniform în $[0, 1]$.

Creaţi un fişier în Python® prin care

(c) să se realizeze N simulări pentru fiecare dintre cazurile (a), respectiv (b);

(d) să se afişeze în aceeaşi figură histrogramele corespunzătoare simulărilor realizate la (c) (pentru fiecare dintre cazurile (a), respectiv (b));

(e) să se afişeze graficul funcţiei de densitate $\lambda e^{-\lambda x}$, $x \in [0, \infty)$;

(f) să se estimeze numeric media şi varianţa variabilei aleatoare distribuită exponenţial $Exp(\lambda)$ folosind simulările de la (c) (pentru fiecare dintre cazurile (a), respectiv (b));

(g) să se afişeze în aceeaşi figură graficul aproximării funcţiei $\mathbb{P}(X \leq x)$ (ca funcţie de $x \in [0, \infty)$) folosind simulările de la (c) (pentru fiecare dintre cazurile (a), respectiv (b)) şi graficul funcţiei $1 - e^{-\lambda x}$, $x \in [0, \infty)$;

```
import numpy as np
import matplotlib.pyplot as plt

# Parametri
lambda_ = 2.0      # λ
N = 10000          # număr de simulări
x_max = 5          # limita superioară pentru grafice

# (a) Generare X folosind funcția Python
X_a = np.random.exponential(1/lambda_, N)

# (b) Generare X folosind formula X = -1/λ * ln(U)
U = np.random.uniform(0, 1, N)
X_b = -np.log(U) / lambda_

# (c) Simulări deja realizate la (a) și (b) în X_a și X_b

# (d) Histogramă
plt.figure()
plt.hist(X_a, bins=50)
plt.hist(X_b, bins=50)
plt.title('Histograme simulări Exp(λ)')
plt.xlabel('x')
plt.ylabel('Densitate')
plt.show()
```

```
# (e) Funcţia de densitate teoretică
x = np.linspace(0, x_max, 500)
#Generăm 500 de puncte între 0 şi x_max.
#Acestea sunt folosite ca axa x pentru a desena graficele funcţiei de densitate şi CDF.
f_x = lambda_ * np.exp(-lambda_ * x)
plt.figure()
plt.plot(x, f_x, 'r', label='f(x) = λ e^{-λx}')
plt.title('Funcţia de densitate a distribuţiei exponentiale')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.show()

# (f) Estimare numerică a mediei şi varianţei
mean_a, var_a = np.mean(X_a), np.var(X_a)
mean_b, var_b = np.mean(X_b), np.var(X_b)
print(f"Metoda Python: media={mean_a:.4f}, varianţa={var_a:.4f}")
print(f"Metoda -1/λ ln(U): media={mean_b:.4f}, varianţa={var_b:.4f}")

# (g) Aproximarea funcţiei de distribuţie
plt.figure(figsize=(10, 6))
# Funcţia empirică
x_sorted_a = np.sort(X_a)
cdf_a = np.arange(1, N+1) / N
x_sorted_b = np.sort(X_b)
cdf_b = np.arange(1, N+1) / N
plt.plot(x_sorted_a, cdf_a, label='CDF simulare Python')
```

```
plt.plot(x_sorted_b, cdf_b, label='CDF simulare -1/λ ln(U)')
# Funcţia teoretică
plt.plot(x, 1 - np.exp(-lambda_ * x), 'r--', label='CDF teoretică 1 - e^{-λx}')
plt.title('Funcţia de distribuţie cumulativă')
plt.xlabel('x')
plt.ylabel('P(X ≤ x)')
plt.legend()
plt.show()
```

EX#4 Durata medie de funcţionare a unui telefon (la capacitatea maximă) este de 2.5 ani, iar a unui alt telefon, de 5 ani. Creaţi un fişier în Python® prin care să se determine şi să se estimeze numeric probabilitatea ca telefoanele să funcţioneze mai mult de 2.5 ani, respectiv 5 ani.

```
import numpy as np

# Datele problemei
mean1 = 2.5
mean2 = 5
N = 100000 # număr simulări

# Parametri distribuţie exponenţială
lambda1 = 1 / mean1
lambda2 = 1 / mean2
```

```
# (1) Probabilitatea exactă folosind formula teoretică
t1 = 2.5
t2 = 5

prob1_theoretical = np.exp(-lambda1 * t1)
prob2_theoretical = np.exp(-lambda2 * t2)

print(f"Probabilitatea teoretică că telefonul 1 funcţionează > {t1} ani: {prob1_theoretical:.4f}")
print(f"Probabilitatea teoretică că telefonul 2 funcţionează > {t2} ani: {prob2_theoretical:.4f}")

# (2) Estimare numerică prin simulare Monte Carlo
X1 = np.random.exponential(mean1, N)
X2 = np.random.exponential(mean2, N)

prob1_sim = np.mean(X1 > t1)
prob2_sim = np.mean(X2 > t2)

print(f"Estimare numerică prin simulare pentru telefonul 1: {prob1_sim:.4f}")
print(f"Estimare numerică prin simulare pentru telefonul 2: {prob2_sim:.4f}")
```

EX#5 Fie $\lambda_1, \lambda_2 \in (0, \infty)$, $X \sim Exp(\lambda_1), Y \sim Exp(\lambda_2)$, X, Y independente, $Z = \min(X, Y)$. Creaţi un fişier în Python® prin care să

(a) să se realizeze N simulări pentru X şi N simulări pentru Y;

(b) să se determine Z folosind simulările de la (a) şi să se afişeze histrograma corespunzătoare;

(c) să se afişeze graficul funcţiei de densitate $(\lambda_1 + \lambda_2)e^{-(\lambda_1+\lambda_2)x}$.

```
import numpy as np
import matplotlib.pyplot as plt

# Parametri
lambda1 = 2.0
lambda2 = 3.0
N = 10000
x_max = 2.0  # pentru grafice

# (a) Simulări
X = np.random.exponential(1/lambda1, N)
Y = np.random.exponential(1/lambda2, N)

# (b) Calcul Z şi histogramă
Z = np.minimum(X, Y)

plt.figure(figsize=(10, 6))
plt.hist(Z, bins=50, density=True, alpha=0.6, color='skyblue')
```

```
plt.title('Histogramă pentru Z = min(X, Y)')
plt.xlabel('z')
plt.ylabel('Densitate')
plt.show()

# (c) Funcţia de densitate teoretică a lui Z
x = np.linspace(0, x_max, 500)
f_Z = (lambda1 + lambda2) * np.exp(-(lambda1 + lambda2) * x)

plt.figure(figsize=(10, 6))
plt.plot(x, f_Z, 'r', label='f_Z(z) = (λ1+λ2)e^{-(λ1+λ2)z}')
plt.title('Funcţia de densitate a lui Z = min(X,Y)')
plt.xlabel('z')
plt.ylabel('f_Z(z)')
plt.legend()
plt.show()
```

#Ce face codul:

#Simulează N valori pentru X şi Y folosind distribuţia exponenţială.

#Calculează Z = min(X, Y) pentru fiecare pereche de simulări.

#Desenează histogramă pentru Z (aproximarea densităţii empirice).

#Desenează funcţia teoretică de densitate pentru Z:

EX#6 Într-o staţie de transport în comun între orele 16 : 00 – 17 : 00, un tramvai ajunge în medie la fiecare 4 minute, iar un autobuz ajunge în medie la fiecare 8 minute. Creaţi un fişier în Python® prin care să se determine şi să se estimeze numeric

(a) probabilitatea ca un călător să aştepte mai mult de 5 minute tramvaiul;

(b) probabilitatea ca un călător să aştepte mai mult de 5 minute tramvaiul sau autobuzul (i.e. probabilitatea ca în primele 5 minute să nu vină niciun vehicul).

Indicaţii Python®: `numpy`, `numpy.random`, `scipy.stats`, `matplotlib.pyplot`, `matplotlib.pyplot.hist`

```
import numpy as np

lambda_tram = 1/4
lambda_bus = 1/8
#Setăm parametrii distribuţiei exponenţiale pentru tramvai şi autobuz.
#Formula:
#λ=1/durata medie
#Deci tramvaiul are λ = 0.25, autobuzul λ = 0.125.
t_wait = 5
N = 100000

# Teoretic
print(np.exp(-lambda_tram * t_wait))
print(np.exp(-(lambda_tram + lambda_bus) * t_wait))

# Simulare
T_tram = np.random.exponential(1/lambda_tram, N)
T_bus = np.random.exponential(1/lambda_bus, N)
```

```
print(np.mean(T_tram > t_wait))
print(np.mean(np.minimum(T_tram, T_bus) > t_wait))
```